

CouncilHound: Turning a City’s Meeting Record into a Searchable, Askable Knowledge Base

Hunter Brown

huntermbrown@gmail.com

councilhound.net

October 2026

describes the system as deployed from `main` (City of Fairfax, Virginia)

Abstract

Local government decisions are made in public, but the public record is hard to use: hours of unindexed video, agendas and minutes scattered across PDF viewers, and project histories that span years of meetings and several boards. CouncilHound is an open pipeline and web application that reads a city’s Granicus meeting archive and its development-project records every night and turns them into a structured knowledge base of meetings, agenda items, roll-call votes, tracked projects and topics, members’ voting records, speaker-attributed transcripts, and project wikis. Residents can browse it, search it, follow it by email, or ask it questions in plain language, and every claim leads back to the source document or the exact second of the city’s own video. Ahead of a city election it also publishes a ballot guide that treats every candidate alike, drawing on dated, hand-verified outside sources. This paper describes what data CouncilHound collects, how each stage of the pipeline processes it, where language models are used and how their output is constrained, and how the results are served. A companion report, *CouncilHound Development Impact Methodology*, documents the economic and fiscal impact models separately.

Contents

1	Motivation and scope	3
2	Design principles	3
3	System overview	4
4	Data sources	4
4.1	Granicus meeting archive	4
4.2	Official development-project records	5
4.3	Geocoding and open data	5
4.4	Election and term data	6
5	The ingestion pipeline	7
5.1	Stage order	7
5.2	Document text extraction	7

5.3	Transcription and speaker attribution	8
5.4	Structuring meetings	9
5.5	Entity identity	9
5.6	Rosters, video timestamps, and profiles	10
5.7	Embeddings	10
6	Data model	10
7	Derived views	10
8	Asking questions: the Ask agent	12
9	Project wikis	14
10	Development impact analysis (summary)	14
11	Delivery: site, feeds, and email	14
12	Operations	15
13	Where models are used, and how they are constrained	16
14	Limitations	16
15	Adapting CouncilHound to another jurisdiction	17
16	Conclusion	17

1 Motivation and scope

A typical small-city council meets twice a month for three hours or more. Its planning commission, school board, and advisory boards add more meetings. A resident who wants to know “what happened with the townhomes on Park Road” has to know which meetings to open, scrub through the recordings, and read packets that can run to hundreds of pages. Professional observers (reporters, neighborhood associations, members themselves) face the same problem at a larger scale.

CouncilHound answers four questions from the record:

1. **What was decided?** Meeting-level ledgers of motions and roll-call votes, with procedural items filtered out.
2. **Where does a matter stand?** A history and current status for every project, ordinance, case, and recurring topic, assembled across meetings and bodies.
3. **Who voted how, and who said what?** Per-member voting records, pairwise agreement, and speaker-attributed transcript passages.
4. **What is coming up?** Upcoming agendas annotated with the tracked matters they name and whether a public hearing is scheduled.

The reference deployment tracks the City of Fairfax, Virginia: its City Council, Planning Commission, School Board, Parks and Recreation Advisory Board (PRAB), and Housing and Healthy Communities Advisory Board (HHCAB). The pipeline is written against the Granicus platform, which hosts meeting archives for hundreds of U.S. local governments, so it can be pointed at another Granicus city with configuration changes (Section 15).

2 Design principles

Five rules shape the system. Most of the implementation choices described later follow from them.

1. **The city’s record is the source of truth.** CouncilHound derives everything from public documents and recordings, and every derived fact keeps a pointer back to them: a document URL, a meeting, an agenda-item label, or a timestamp.
2. **Video is linked, never hosted.** Every “watch” affordance is a deep link into the city’s own Granicus player, seeked to the relevant second. CouncilHound never re-hosts, embeds, or proxies meeting video.
3. **Language models never own identity.** Models extract *names*, and code decides which entity a name refers to, when two entities are the same, and who a speaker is allowed to be named as. A model’s output can never create or merge a record’s identity on its own.
4. **Constrain, verify, and record model output.** Wherever a model is used, its output is shaped by a forced tool schema, checked in code against the source text, and stored raw with a prompt version so it can be re-applied or audited without another call.
5. **Every stage is idempotent.** Each pipeline stage can be re-run at any time and converges on the same state. That makes nightly re-processing, backfills, and recovery from partial failure routine operations rather than incidents.

3 System overview

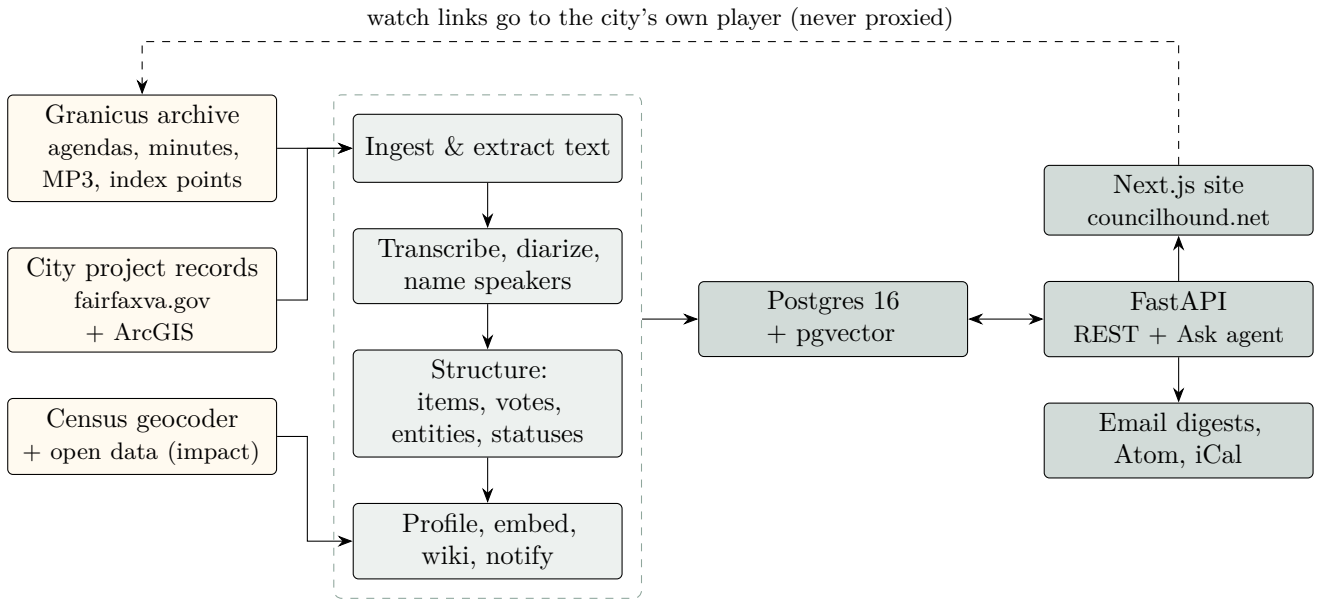


Figure 1: Data flow. Ingestion stages (green, in the dashed box) run as a scheduled batch job; the API and web front end are read-mostly services over the same database.

CouncilHound has three parts (Figure 1):

- an **ingestion pipeline** (Python 3.12, SQLAlchemy 2, Alembic) exposed as a command-line tool whose **daily** command chains every stage;
- a **PostgreSQL 16** database with the **pgvector** extension, holding both relational records and vector embeddings;
- a **FastAPI** service and a **Next.js 14** front end that serve the public site, the plain-language question-answering agent, and the feeds.

Production runs on Fly.io (Section 12).

4 Data sources

4.1 Granicus meeting archive

Granicus is the meeting-video platform behind most U.S. municipal “watch meetings online” pages. CouncilHound reads it as an anonymous web visitor would. Every request carries a browser user agent (Granicus refuses bare clients), is spaced at least one second apart, and is retried with back-off on server errors and throttling.

Granicus serves caption files for these meetings, but they are empty, so CouncilHound transcribes the audio itself (Section 5.3). Granicus’s content network also blocks requests from data-center IP ranges for media files. Audio is therefore downloaded and transcribed on a residential machine, while everything else runs in the cloud.

Endpoint	What CouncilHound takes from it
<code>ViewPublisher.php</code>	The archive listing: one row per recorded meeting with date, duration, clip identifier, agenda and minutes links, MP3/MP4 links, and a document drop-down (actions reports, minutes, other). The same page carries the <i>upcoming events</i> table.
<code>AgendaViewer.php</code>	The agenda, either HTML or an uploaded PDF or Word file; through GeneratedAgendaViewer , links to per-item attachments.
<code>MetaViewer.php</code>	Per-agenda-item attachments: staff reports, draft motions, resolutions, plans.
<code>MinutesViewer.php</code>	Minutes and the clerk’s actions report, which are the authoritative record of motions and votes.
<code>MediaPlayer.php</code>	The clerk’s <i>index points</i> : per-agenda-item timestamps in the recording.
<code>archive-video.granicus.com</code>	Meeting audio (MP3) for transcription.

Table 1: Granicus endpoints used.

Each archive section is mapped to a tracked *body*. Within a body, a meeting’s title sets its type (regular, work session, special, retreat, closed session). Rows with no recording, such as cancelled or future meetings, are skipped, and cancelled-meeting clips are never transcribed because their title-card music makes speech recognition hallucinate. The two advisory boards publish agendas and minutes but no recordings.

4.2 Official development-project records

The city publishes a directory of major development and capital projects. CouncilHound reads it from two places:

- the **city website’s project pages**, which give project type, division, official schedule and status, timeline, background, the applicant’s requests, linked documents, address and coordinates, assigned planner, and project image;
- the city’s **ArcGIS feature service** for the major-developments map, which gives name, description, applicant, development status (pre-application, under review, approved, under construction), project URL, and point geometry.

The city website also blocks data-center IPs, so cloud runs fall back to the ArcGIS layer alone. A partial run never overwrites richer fields gathered earlier and never deletes a record. Each official project becomes a **project** entity, and the meetings that discuss it are linked to it through the same entity-resolution path used for meeting-derived topics (Section 5.5). The site currently carries 41 official project records.

4.3 Geocoding and open data

Locations named in meetings (streets, addresses, sites) are geocoded with the U.S. Census Bureau’s public geocoder, constrained to the city. Failures are recorded so they are not retried every night. The development-impact subsystem draws on further open datasets: American Community Survey block groups, TIGER geography, LEHD origin–destination employment data, OpenStreetMap street networks, merged Overture, Foursquare, and OpenStreetMap points of

interest, transit GTFS feeds, city GIS layers, and parcel assessment records (owner names are never stored). Those sources and models are covered in the companion methodology report.

4.4 Election and term data

Member terms, seat expirations, and the current ballot (candidates by seat) are pinned by hand from the city’s and registrar’s published sources. Each entry records its source URL and the date it was verified. The data powers term and ballot answers in the question-answering agent, and an election date stops being reported once it has passed.

Candidates. Most candidates have never voted at a meeting the record covers, so the record alone can say nothing about them. A hand-curated table holds, for every name on the official sample ballot, outside sources of six kinds: campaign sites, voter guides, campaign finance reports, official rosters and profiles, candidate forums, and local news. Each source keeps its URL, publication date, and the date its facts were checked. Facts are short neutral paraphrases attributed to the source (“The campaign site lists ...”, “Patch reports ...”); a campaign’s claim is never stated as fact. The table follows fixed rules:

- every candidate in a contest is looked up the same way, and what was looked for and *not* found is recorded too, so a gap reads as a gap rather than as silence;
- donors are named only when they are public officials or organizations;
- details of spouses, parents, and children are left out;
- a source owned by a candidate is not used.

A test ties the table to the ballot: every name on the ballot has exactly one entry.

Questionnaires. The voter-guide questionnaires (the League of Women Voters’ Vote411 and a local news outlet’s) put the same questions to everyone in a race. They are kept answer by answer so the answers to one question can be read side by side. Only questions every candidate in the race was asked are kept. Each answer is a paraphrase of at most 280 characters and never longer than the answer itself, so a terse answer stays terse. A question left blank is kept as blank. The raw answers are extracted by script, paraphrased in one pass, and then checked against the published text by a separate model pass that did not write them; on the first build that check corrected 14 of 94 paraphrases, mostly for dropped hedges (“seems”, “projected”) and truncated proposal names. A test keeps the candidate table’s “not yet responded” entries consistent with the questionnaires.

Keeping it current. A scheduled task runs each morning until election day. It looks for new voter-guide answers, questionnaires, finance reports, and coverage, verifies each fact against the page itself, has new questionnaire paraphrases checked independently, and opens a pull request. A person reviews and merges every change; the task never deploys. After the election the table is retired with the ballot.

5 The ingestion pipeline

5.1 Stage order

The `daily` command runs the stages below over a rolling 14-day window. A lighter `catchup` command (5-day window, no audio) runs hourly, so minutes and actions reports posted between nightly runs are picked up quickly.

1. **Ingest**: scrape the archive and upsert meetings and documents, then download audio.
2. **Upcoming**: replace the upcoming-events table and fetch each posted agenda's text.
3. **Projects**: sync official project records.
4. **Extract text**: turn PDF, HTML, and Word documents into text.
5. **Transcribe**: speech-to-text, speaker separation, timed chunks.
6. **Structure**: model-assisted extraction of agenda items, votes, entities, and statuses.
7. **Name speakers**: attribute diarized speakers from cues in the transcript.
8. **Voice match**: carry confident speaker names across meetings by voice.
9. **Index points**: attach official video timestamps to agenda items.
10. **Seed roster**: parse member rosters and titles from agenda headers.
11. **Deduplicate**: merge entities that differ only by naming drift.
12. **Geocode**: place location entities on the map.
13. **Profile**: refresh topic summaries whose underlying record changed.
14. **Embed**: compute vector embeddings for new transcript chunks and agenda items.
15. **Notify**: send confirmed followers their email digests.

Every stage is also a separate CLI command, so a single stage can be re-run for a single meeting. The expensive stages (ingest, transcribe, structure, speaker naming, voice matching, embed, profile) each take a non-blocking PostgreSQL advisory lock. If a nightly run, the hourly catch-up, and a one-off backfill overlap, a runner that finds a stage taken skips it rather than duplicating paid model calls. Postgres releases the locks automatically when a connection drops, so a killed machine never leaves a stale lock behind.

5.2 Document text extraction

A document's format is identified from its content type and, because Granicus labels Word files as `application/msword`, by sniffing its leading bytes. Text is then extracted:

- **PDF**: with PyMuPDF;
- **HTML**: with BeautifulSoup, with scripts and styles removed;
- **Word (.docx)**: from the document XML, using only the Python standard library (closed-session agendas are often posted this way).

Extracted text is sanitized. NUL bytes and control characters are stripped, and a document whose text is less than 85% printable characters (a symptom of a broken font encoding) is treated as having no text, not as garbage text. Scanned documents with no text layer are kept with empty text and flagged for a later OCR pass.

5.3 Transcription and speaker attribution

Speech to text. Audio is transcribed with OpenAI’s Whisper models, using `mlx-whisper` (`whisper-large-v3-turbo`) on Apple-silicon GPUs and `faster-whisper` on CPUs. Decoding is fixed to English, emits word-level timestamps, does not condition on previous text, and suppresses output over long silences. These settings guard against the repetition and hallucination loops that long civic recordings otherwise produce.

Speaker separation. Speaker diarization uses `pyannote’s speaker-diarization-community-1` model. Each recognized word is assigned to the speaker turn it overlaps most. The transcript is then cut into chunks of about 700 characters that never cross a speaker change, and each chunk keeps its start and end time and an anonymous speaker label such as `SPEAKER_05`. Older meetings were re-transcribed with speaker labels. A re-transcription is accepted only if it keeps at least 90% of the original’s words.

Naming speakers. Anonymous labels become names in one constrained model call per meeting. The model sees the labelled transcript and a *roster* assembled in code:

- members who voted in that body within 90 days of the meeting;
- holders of that body’s titles;
- people already named in the meeting’s record.

It may assign a name only on a cue spoken in the transcript: a self-introduction, being recognized by the chair, or being addressed by name. For each assignment it must quote the cue. Code then verifies every quote:

- the quote must appear in the transcript within 90 seconds of its claimed time, next to that label’s own speech;
- a “high” confidence claim without verified evidence is downgraded;
- a name outside the roster must literally occur in the transcript.

A name is published only at high confidence on a label that does not mix several voices. Student representatives are labelled “Student” and are never named.

Voice matching. Confidently named speakers are carried across meetings by voice. For each speaker label, the system averages up to six 10-second `pyannote` embeddings into a 256-dimensional fingerprint. A member’s voiceprint is built only from their named appearances in *other* meetings (leave-one-meeting-out), and requires at least two such meetings. An unnamed label is matched to a member only if all of these hold:

- cosine similarity of at least 0.80;
- a margin of at least 0.25 over the next-best candidate;

- at least 20 seconds of speech.

Voice evidence never overrides a name spoken in the transcript, a manual correction, a student, or a mixed label. When voice and transcript disagree, the disagreement is reported, not applied. As of this writing about 91% of transcript chunks in production carry a speaker name.

5.4 Structuring meetings

The structuring stage turns each meeting’s documents into records. It sends the agenda, the minutes, and the actions report (each truncated to 60,000 characters) to Claude (Anthropic) with a single forced tool, `record_meeting_extraction`. The tool’s schema is:

- a meeting **summary**;
- **agenda items**: label (e.g. 7a), title, description, outcome, and
 - **votes**: motion text, result (`passed/failed/deferred`), and a breakdown mapping each member’s surname to `yes/no/abstain/absent`;
 - **entities**: name, type (`project, ordinance, resolution, case_number, location, topic, person`), role, and the status the item left it in (`proposed, in_progress, approved, denied, deferred, completed, withdrawn`);
- **other discussion** outside numbered items (public comment, reports, announcements).

Transcripts are deliberately *not* used for structuring. Decisions come from the clerk’s written record, which is authoritative. The prompt also lists up to 60 already-tracked entity names that appear in the documents and tells the model to reuse them exactly, so a multi-year matter keeps one name.

Code then applies guards to the model’s output:

- **No record, no votes.** Votes are kept only when minutes or an actions report with text exist. Given only an agenda packet, a model can mistake sample motions for actual ones.
- **Unanimous votes are expanded only from attendance.** When minutes record “passed unanimously” over an attendance list, each member recorded present is counted yes (no, for a unanimous failure) and absentees absent. Nothing is inferred if neither a roll call nor attendance is written down.
- **The city is never a topic.** The jurisdiction and its own boards are filtered out as entities.
- **Late records re-trigger extraction.** A meeting structured from its agenda alone is re-extracted, exactly once, when its minutes or actions report arrive.

The raw model output is stored per meeting and prompt version, and applying it deletes and recreates that meeting’s derived rows. Re-running structuring is therefore idempotent, and a change to the application logic can be replayed over stored outputs without calling the model again.

5.5 Entity identity

An *entity* is anything the record tracks across meetings: a project, ordinance, resolution, case number, location, recurring topic, or person. A name is resolved to an entity in code, in this order:

1. **exact slug:** the name is normalized (case, punctuation, a trailing “project(s)”, a leading article, abbreviations such as *Blvd/Boulevard*);
2. **alias:** a case-insensitive match against recorded alternate names;
3. **drifted base:** if stripping a descriptive suffix (*development, redevelopment, review, zoning, update*, a year, a trailing acronym) yields an existing entity of the same type, the new name becomes an alias of that entity;
4. otherwise a new entity is created.

A nightly deduplication pass and an operator merge command catch what slips through. Merges move all history and keep the old name and URL slug as aliases, so links never break. Each entity records one *update* per meeting (what happened and the status after) plus finer-grained *mentions*. Its current status is the latest status set, ordered by meeting date.

5.6 Rosters, video timestamps, and profiles

Rosters. Member rosters are parsed from agenda headers: the council’s Mayor and Council blocks, the Planning Commission’s Chair/Vice-Chair/Commissioners line, and the School Board’s two-names-per-line block. Each member gets body-qualified title aliases (“Councilmember ...”, “School Board Chair ...”). A member counts as sitting if they appear in the most recent parseable agenda header for their body.

Video timestamps. The clerk’s index points are matched to agenda-item labels, so each item, vote, and timeline entry can link to the moment it begins. Links pass both **starttime** (legacy player) and **entrytime** (current player) so they seek correctly in either Granicus player.

Topic profiles. For each non-person entity with at least two updates, a model call writes a short summary, a list of open questions, and per-member commentary. The input is the dated history with outcomes and votes plus up to a dozen transcript excerpts that mention the entity. Minutes take precedence over transcripts for vote counts. A profile records the meeting it is current through and is rebuilt only when a newer update arrives or the prompt version changes.

5.7 Embeddings

Transcript chunks and agenda items (label, title, description, outcome) are embedded with BAAI/**bge-base-en-v1.5**, a 768-dimensional open sentence-embedding model run locally on CPU, with normalized vectors. Queries are embedded with BGE’s retrieval-instruction prefix, and stored passages without it. Vectors live in **pgvector** columns beside the rows they describe and are searched by cosine distance. Trigram indexes support keyword search over the same text.

6 Data model

7 Derived views

Most of what the site shows is computed deterministically from the tables above, without a model in the loop.

Table	Contents
<code>meetings</code>	One recorded meeting (Granicus view and clip), body, type, date, duration.
<code>documents</code>	Agendas, minutes, actions reports, item attachments; source URL and extracted text.
<code>agenda_items</code>	Items per meeting: label, title, description, outcome, video offset, embedding.
<code>votes</code>	Motion, result, and member-by-member breakdown.
<code>transcript_chunks</code>	Timed ~700-character passages, speaker label and resolved speaker, embedding.
<code>meeting_speakers</code>	Per-meeting speaker label → name, role, confidence, quoted evidence, source (model, voice, manual).
<code>speaker_voices</code>	256-dimensional voice fingerprint per meeting and label.
<code>entities, entity_aliases</code>	Tracked matters and people; canonical slug, type, current status; alternate names and retired slugs.
<code>entity_updates, entity_mentions</code>	One update per entity per meeting with status after; finer-grained mentions.
<code>entity_profiles</code>	Cached model-written summaries, open questions, member commentary.
<code>entity_geocodes</code>	Coordinates from the city record or the Census geocoder.
<code>city_projects</code>	Official project records from the city website and GIS.
<code>upcoming_meetings</code>	Posted future meetings and their agenda text.
<code>extractions</code>	Raw structuring output per meeting and prompt version.
<code>wiki_pages</code>	Database mirror of the project-wiki bundle.
<code>project_evaluations</code>	Impact-analysis specifications, results, and reports.
<code>topic_subscriptions</code>	Email follows and their delivery watermarks.
<code>ingest_runs</code>	Per-run bookkeeping and error counts.

Table 2: Principal tables.

Hot topics: named discussion time. A topic’s prominence is the total duration of transcript chunks whose text contains its name or one of its aliases (five or more characters), summed over a body’s meetings in a window (60 days by default). This measures how much floor time a matter received rather than how often it was listed. It is a lower bound by construction, because references by pronoun are not counted.

Member records. Each recorded vote is credited to a member by surname, including the final part of a hyphenated name. From these votes the system computes:

- a yes/no/absent split;
- the share of votes cast with the outcome;
- contested votes (at least one no), minority votes, close votes, and lone dissents;
- agreement with each sitting colleague on contested votes.

Comparison panels appear only once a member has at least ten contested votes, because smaller samples are not informative.

Change feed. Status transitions and first appearances in a window are measured against the last status actually set, not the previous row. The same logic feeds the site’s “changed this

week” ledger, an Atom feed, and the weekly email briefing.

Pre-meeting briefs. Each posted upcoming agenda is matched against tracked entity names and aliases, and public-hearing sections are detected from the agenda’s numbered headings. The result is an annotated agenda: each tracked matter with its current status, what happened last time, and links to its history and impact analysis.

Near me. Projects and named places within a chosen radius of an address (geocoded through the Census geocoder) or of the browser’s location, nearest first.

8 Asking questions: the Ask agent

The `/ask` endpoint answers plain-language questions about the record. It is the only endpoint that makes a model call for each request. Rather than a single retrieve-then-answer step, it is a small **tool-using agent**.

Opening context. Before the model runs, code assembles:

- today’s date and the span of the indexed record;
- the topics and members the question names outright;
- a first hybrid search over the record.

A simple question can often be answered with no further calls.

Tools. The model may then call up to five rounds of tools:

Tool	Returns
<code>search_record</code>	Hybrid keyword and semantic search over transcripts and agenda items, with roll calls, filterable by body and date.
<code>search_documents</code>	Exact-phrase search over the full text of agendas, minutes, and staff reports.
<code>get_topic</code>	A topic’s profile, wiki pages (labelled with their review status and staleness), official record, impact summary (labelled as modelled estimates), timeline, and roll calls.
<code>get_member</code>	A member’s voting record, term, position statements, and contested votes.
<code>get_statements</code>	What a member said, in their own words, from speaker-attributed transcript passages.
<code>compare_members</code>	Pairwise agreement among two to six members and the roll calls where they split.
<code>list_members</code>	Sitting members with term end and ballot status, and the official candidate list.
<code>get_candidate</code>	One candidate, or everyone in a contest in ballot order, with the outside sources on file and what was looked for and not found (Section 4.4).
<code>get_upcoming</code>	Upcoming meetings and their agenda text.
<code>search_web</code>	Verbatim passages from a short allowlist of local government, school, election, and news sites, for what the record does not hold.

Table 3: Ask agent tools.

Hybrid retrieval. `search_record` merges two lists:

- a *keyword* pass (substring match over items and transcript chunks, newest first), which reliably finds proper nouns and case numbers;
- a *semantic* pass (pgvector cosine search over the BGE embeddings, kept only within a distance threshold of 0.55), which finds paraphrases.

The merged list is deduplicated and capped at ten results.

Citations by construction. Every record any tool returns is registered once per question under a stable number, and those numbers are shared across tool calls. The model cites by number. After the answer is written, code turns only the numbers that exist in the registry into links, each to a document, a meeting item, or a timestamp in the video. A number the model makes up stays plain text and is never linked. The answer is displayed beside a rail of its sources in date order, with cards for the topics and members it concerns.

Outside the record. Candidate sources and web passages are cited like everything else but are labelled *outside the record* wherever they appear. `search_web` is built so that nothing reaches an answer that a page did not say word for word. Two small model calls (Claude Haiku) do the reaching:

1. a server-side web search, limited to sixteen allowlisted domains, whose prose is discarded and whose citations only say which pages matter;
2. a server-side fetch of the top two cited pages in full.

Code then splits each page into passages of about 700 characters and keeps the four closest to the query by the same embedding model the record search uses (cosine similarity of at least 0.5), verbatim. When a page cannot be fetched, its search snippets stand in. Campaign sites are not on the allowlist (the curated table covers them), nor is a news outlet owned by a candidate. The agent is told to search when the record’s latest word on a current matter is old or tentative.

Follow-ups. A question may carry up to four earlier turns of the conversation, so “what did she say about it?” resolves against the thread. Earlier answers are context, not sources: anything restated must be cited again from a source retrieved in the current turn. After each answer the agent suggests up to three short follow-up questions that the record it was shown could likely answer; it may not suggest what its answer says the record lacks, nor ask for motives. An answer that cites candidate sources also links to the election page.

Model and limits. The agent runs on Claude Sonnet with prompt caching on its system prompt. It streams its progress (“Pulling the voting record...”) to the browser. Questions are limited to 500 characters. The endpoint is rate-limited per client IP (six questions per minute by default) and by a global daily budget (500 by default), which bounds both abuse and cost.

Site search. The non-agent search page uses the same hybrid approach without a model. It leads with tracked topics whose names or aliases match, then lists keyword and semantic hits from transcripts and agenda items, each linked to its moment in the video.

9 Project wikis

For each official project, CouncilHound maintains a small wiki in the Open Knowledge Format (OKF) v0.2. OKF is a convention for durable markdown knowledge bundles with YAML front matter. Each project’s wiki has an overview, a meeting history, a page of members’ and the public’s positions, and an impact page. The bundle is version-controlled, so every change is a reviewable diff. Ownership is per file:

- **Pipeline-owned** pages (history, indexes, status front matter) are regenerated deterministically from the database.
- **Curator-owned** pages (overview, positions, impact) are seeded once. After that, a model curator makes *minimal* edits when new meetings arrive. It sees the current page and only the record newer than the page. Regions marked **curator:off** are untouchable, and an edit that changes one is rejected whole. Human edits therefore survive.
- **Numbers are never written into prose.** Impact figures appear as `{{metric:...}}` markers that the site resolves against the live evaluation, so a page cannot carry a stale number. A linter checks every marker and link before a change is committed.

Each page carries OKF’s trust metadata:

- **who generated it:** a deterministic process or a named model;
- **when:** the meeting date the page is current through;
- **when it may be outrun:** the next scheduled meeting of a body that has taken up the project;
- **sign-off:** any human verification events.

The site renders these as “reviewed”, “not yet reviewed”, “edited since review”, and “a meeting may have outrun this page” badges.

10 Development impact analysis (summary)

For official development projects, CouncilHound publishes screening-level economic and fiscal impact estimates. A model extracts project facts (units, square footage, tenure, parcels) from public documents. Each fact must be supported by a short verbatim quote that code verifies against the source, and a human confirms the specification before any numbers are computed. Deterministic modules then compute retail capture, walking activity, employment, and fiscal revenue and service-cost ranges over open data. Every assumption is named, bounded, and propagated, and the project page lets readers adjust the assumptions and recompute the estimates in the browser. A model writes the narrative afterwards, and a validator requires every number in it to match a computed output. The formulas, calibration against city staff estimates, and limitations are documented in the companion report.

11 Delivery: site, feeds, and email

The public site (councilhound.net) has these surfaces:

- a **briefing** home page, whose lede picks, in order, an upcoming public hearing, the most recent contested vote, the next meeting with tracked business, or a summary count;
- **meetings** as a month-by-month ledger of decisions, each meeting page leading with what was decided, its documents, and chaptered watch links;
- a **projects and topics** directory with facets, a map, and a page per topic or project;
- **members** with voting records;
- an **election** page while a city election is ahead: every contest on the official sample ballot as a tab, every candidate in ballot order on an identical card (the same checklist of source kinds, “none found” where a kind is missing, and what was looked for and not found), a link to the voting record of any candidate who already holds office, and the questionnaire answers set side by side question by question. Readers can narrow the comparison to the candidates they are weighing; nothing is ranked, scored, or summarized;
- **search**, **Ask**, **near me**, and a **glossary** of agenda vocabulary that also appears as hover definitions across the site.

Readers can **follow** a topic, a member’s votes, a body’s meetings, an area of the city, or a weekly briefing by email. Follows are confirmed by a tokened link, each person receives one digest combining all their follows, and every digest has a one-click unsubscribe. Delivery watermarks advance only after a successful send, so a failed night is retried, not skipped. An iCalendar feed of upcoming meetings and an Atom feed of status changes serve calendar and feed readers.

12 Operations

Component	Deployment
Web	Next.js on Fly.io (shared CPU, 512 MB); API responses cached for five minutes.
API	FastAPI on Fly.io (2 shared CPUs, 4 GB); the embedding model is baked into the image and loaded at startup.
Database	Self-run <code>pgvector/pgvector:pg16</code> machine with a persistent volume (Fly’s managed Postgres lacks <code>pgvector</code>).
Jobs	A scheduled machine runs <code>daily</code> once a night and stops; a second runs <code>catchup</code> hourly.
Local	A residential machine downloads and transcribes audio overnight (Granicus blocks cloud IPs), names speakers, and syncs the project wikis to production.

Table 4: Production deployment (Fly.io, Ashburn region; DNS on Cloudflare).

Testing. Continuous integration runs the ingestion and API test suites against a real `pgvector` database, with model and embedding calls mocked. It also exercises schema migrations up, down, and up again, and type-checks and builds the front end. A separate weekly *canary* parses the live Granicus archive and upcoming-events table and asserts minimum counts of meetings, agendas, audio links, index points, and events. It catches markup drift that tests pinned to saved fixtures cannot.

13 Where models are used, and how they are constrained

Task	Model’s job	Constraint in code
Meeting structuring	Read documents into items, votes, entities, statuses	Forced tool schema; votes only with minutes; names resolved to entities by code; raw output stored by prompt version
Speaker naming	Name diarized labels from spoken cues	Roster built by code; quoted evidence verified by time and position; publish only at high confidence; students never named
Topic profiles	Summarize a matter’s history	Built only from the stored record; minutes outrank transcripts; regenerated only when the record changes
Wiki curation	Make minimal edits to curated pages	Protected regions; metric markers instead of numbers; lint before commit
Impact extraction / narrative	Extract project facts; write the report	Verbatim-quote firewall; human confirmation gate; validator ties every number to a computed output
Ask agent	Choose tools and compose an answer	Tools return registered sources; only registered citations are linked; rate and budget limits
Web lookup	Find and read pages on allowlisted sites	Search prose discarded; only passages quoted verbatim and selected by embedding similarity are returned; labelled outside the record
Candidate curation	Paraphrase outside sources and questionnaire answers	Every fact checked against its page; questionnaire paraphrases checked by a separate pass; length capped by test; changes reviewed by a person

Table 5: Model use across the system. Speech recognition, diarization, voice matching, and embeddings use open models running locally; the tasks above use Claude.

No model output is treated as a source in its own right. Each model call either produces a candidate that code checks against the record, or rewrites the record in plainer words while linking back to it.

14 Limitations

- **Coverage depends on the city’s record.** Advisory boards without recordings have no transcripts. Meetings whose minutes are late carry no votes until the minutes arrive. Scanned documents without a text layer await OCR.
- **Surname-keyed votes.** Breakdowns are keyed by surname as minutes record them. Two

members of one body who share a surname would need disambiguation that the current schema does not provide.

- **Extraction errors are possible.** The guards above reduce, but cannot eliminate, mis-read motions or mis-assigned statuses. Every derived record links to its source so readers can check.
- **Speaker attribution is conservative.** Roughly one chunk in ten remains unnamed by design rather than guessed.
- **Named discussion time is a lower bound** and depends on how consistently a matter is named aloud.
- **Impact estimates are screening estimates**, not forecasts. See the companion report.
- **Outside sources age.** Candidate information is pinned by hand and is only as current as its last check; every source shows that date. Questionnaire paraphrases are summaries, and each links to the candidate's full answer.

15 Adapting CouncilHound to another jurisdiction

Granicus archive pages share one skeleton across cities. Porting CouncilHound requires:

1. the city's Granicus subdomain and archive view identifiers;
2. a mapping from the city's archive section names to the bodies to be tracked (the body registry);
3. roster parsers matched to the city's agenda-header format;
4. for project tracking and impact analysis, a jurisdiction file of FIPS codes, GIS layer URLs, and pinned tax and budget rates, each with its source and fiscal year. Metrics that depend on an unpinned rate refuse to run rather than guess.

Work is under way to make one deployment per jurisdiction fully configuration-driven, starting with Fairfax County.

16 Conclusion

CouncilHound treats a city's public record as primary data. It collects that record faithfully, structures it with models that are constrained and checked in code, and always links back to the original document or moment of video. The result is a record residents can search, follow, and question, where the provenance of every answer is visible and can be checked.